10f. SIMD: Branchless Code

Martin Alfaro
PhD in Economics

INTRODUCTION

SIMD accelerates workloads by applying a single instruction to many elements at once. Critically, its effectiveness hinges on all elements following the same control flow. Yet, this pattern can be disrupted when conditionals are introduced, either explicitly in the source code or implicitly by the compiler.

Once data elements evaluate a condition differently, their execution paths diverge and the hardware can no longer maintain a unified instruction stream. While compilers attempt to mitigate this issue by transforming code into SIMD-friendly forms, these adaptations tend to introduce additional work. This added overhead reduces the net benefit of SIMD.

The current section explores strategies for efficiently applying SIMD in the presence of conditional behavior. In particular, we'll show how to restructure computations to make control-flow branches disappear, allowing the program to preserve a uniform instruction stream.

We'll first examine scenarios in which the compiler introduces branches as a byproduct of its internal computation techniques. In these situations, users may be unaware that conditional control flow has been generated. For this reason, it's essential to recognize when such hidden branches occur.

After this, we consider explicit conditional statements written directly by the user. These conditions are intrinsic to program logic, ensuring that certain computations occur only when specific criteria are met. Because they can't simply be eliminated, we revisit common approaches to expressing conditions and analyze their internal implementation. The aim is to compare their relative strengths and limitations, indicating which ones interact more favorably with SIMD execution.

Finally, we show that conditional statements can often be reformulated as algebraic operations. This transformation removes branching logic and restores SIMD's ability to operate on a uniform instruction stream. In fact, compilers may implement these transformations automatically when it's likely beneficial.

HIDDEN BRANCHING

Certain coding patterns lead to hidden branching. These are conditional execution paths that the user doesn't write explicitly, but the compiler adds internally to carry out the computation.

TYPE INSTABILITY

One particular source of hidden branching is type instability. When a function can return values of different types, Julia must generate separate execution paths to handle each possibility. These extra branches remain invisible in the source code, yet they still break the uniform instruction flow that SIMD depends on. The solution is simply to restore type stability, for which we can apply all the techniques explored in Chapter 8 of this book.

It's worth remarking, though, that ensuring type stability should always be a priority regardless of any SIMD consideration: any attempt to achieve high performance is nearly impossible without type stability. Considering this, we'll proceed throughout the section assuming that type stability was guaranteed.

BOUNDS CHECKING

A second source of hidden branching arises in for-loops due to bounds checking. This consists of executing each iteration only when indices remain within valid bounds,representing a subtle form of conditional execution. Bound checks interfere with the application of SIMD, as it prevents the compiler from treating the for-loop as a uniform sequence of operations.

This is why disabling bounds checking by **adding `@inbounds` is a prerequisite for the application of SIMD**. If bound checks remain in place, inserting `@simd` alone will rarely lead the compiler to emit SIMD instructions. In fact, the opposite usually occurs: merely adding `@inbounds` may be sufficient to enable SIMD on its own. In those cases, you'll notice that using `@inbounds @simd` rather than `@inbounds` won't yield any additional speedup. Both of these points are demonstrated below. [1]

NO MACROS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.392 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.384 ms (3 allocations: 7.629 MiB)
```

@INBOUNDS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.226 ms (3 allocations: 7.629 MiB)
```

@INBOUNDS + @SIMD

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  5.382 ms (3 allocations: 7.629 MiB)
```

A practical lesson from these examples emerges: when signaling to the compiler that SIMD is appropriate, for-loops should be annotated with `@inbounds @simd` rather than `@simd` alone.

Broadcasting and For-Loops

Broadcasting in Julia automatically disables bounds checking and applies SIMD instructions. This often makes broadcast versions appear faster than a straightforward for-loop. However, **broadcasting is just compact notation for implementing for-loops**. As a result, a for-loop annotated with `@inbounds @simd` usually achieves a similar performance to its broadcast variant, as demonstrated below.

BROADCASTING

```
x      = rand(1_000_000)
foo(x) = 2 ./ x
```

```
julia> @btime foo($x)
  1.190 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (NO MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.384 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (OPTIMIZED WITH MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.191 ms (3 allocations: 7.629 MiB)
```

APPROACHES TO CONDITIONAL STATEMENTS

Certain workflows require the introduction of conditional statements. When this occurs, conditions are integral to a program's logical flow and can't be avoided. It's therefore important to consider the most effective way to express them.

In this book, we've examined several ways to introduce **conditional statements**. In the context of SIMD, a relevant distinction among these approaches is whether they evaluate conditional statements **eagerly or lazily**.

To illustrate, let's consider computing `1 + 1` only when certain condition `C` is met. A lazy strategy checks `C` first, performing the addition `1 + 1` only if the condition is true. In other words, the operation is deferred until it's actually needed. By contrast, an eager approach performs `1 + 1` immediately, regardless of whether `C` is satisfied. If the condition eventually evaluates to `false`, the result of `1 + 1` is simply discarded.

When a conditional statement is applied only once, a lazy approach is more performant because it avoids unnecessary work. Instead, a trade-off arises when the conditional appears inside a for-loop, where its evaluation may repeatedly apply. In this case, SIMD can apply the same operation across many elements at once, making it advantageous to compute all branches upfront and then select the relevant results only afterward. This is especially true when the branches involve inexpensive computations.

In Julia, whether a conditional expression is evaluated lazily or eagerly depends on how the code is written. Next, we'll examine this aspect in more detail.

IFELSE VS IF

The `ifelse` function in Julia follows an eager evaluation strategy. This means that both the condition and all possible outcomes are computed before deciding which result to return. In contrast, the `if` construct favors lazy computations, only evaluating the branch corresponding to the condition's truth value.

The following example demonstrates the computational difference between `if` and `ifelse`. The example is based on a reduction operation, where the elements to be summed are contingent on a condition. Because `ifelse` requires specifying an operation for both the true and false cases, we return zero when the condition isn't met.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.312 ms (0 allocations: 0 bytes)
```

IF + SIMD

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.321 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  1.221 ms (0 allocations: 0 bytes)
```

IFELSE + SIMD

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  349.996 µs (0 allocations: 0 bytes)
```

As the example reveals, the fact that `ifelse` is eager doesn't automatically trigger the application of SIMD. This is precisely why `@inbounds @simd` had to be included.

Note that applying SIMD instructions with `ifelse` doesn't necessarily increase performance. The example below demonstrates this point.

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  56.122 ms (0 allocations: 0 bytes)
```

IFELSE + SIMD

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  50.983 ms (0 allocations: 0 bytes)
```

TERNARY OPERATOR

Ternary operator is an alternative approach to expressing conditional logic. It follows the syntax `<condition> ? <action if true> : <action if false>`. Unlike the previous methods, this form doesn't enforce an eager or a lazy approach. Instead, it relies on heuristics to determine which approach should be implemented. The decision depends on the compiler's assessment about which strategy will likely be faster in the specific application.

To illustrate this behavior, we revisit the same reduction example used earlier, now applying `@inbounds @simd` directly within each construct. In the following case, the ternary operator chooses an eager strategy, thus evaluating both expressions before selecting the relevant branch.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.318 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i]>0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> foo!($x)
  386.313 μs (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.5 ? x[i]/2 : 0
    end

    return output
end
```

```
julia> foo!($x)
  392.661 μs (0 allocations: 0 bytes)
```

Instead, the ternary operator opts for a lazy approach in the following example.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.99
            output += log(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.267 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.99, log(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  11.496 ms (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.99 ? log(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  1.358 ms (0 allocations: 0 bytes)
```

It's worth noting that the method chosen by **the ternary operator isn't foolproof: it could choose a less performant approach**. In the following example, the ternary operator ends up implementing the slower approach.

IF

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output[i] = log(x[i])
        end
    end
end
```

```
julia> foo!($output,$x)
  79.400 ms (0 allocations: 0 bytes)
```

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, log(x[i]), 0)
    end
end
```

```
julia> foo!($output,$x)
  54.072 ms (0 allocations: 0 bytes)
```

TERNARY

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = x[i]>0.5 ? log(x[i]) : 0
    end
end
```

```
julia> foo!($output,$x)
  80.596 ms (0 allocations: 0 bytes)
```

WHEN IS EACH APPROACH BETTER?

As a general guideline, conditional statements with **computationally-demanding operations will more likely benefit from a lazy implementation**. In contrast, **branches comprising simple algebraic computations tend to benefit from an eager approach**. These heuristics tend to drive the decision adopted by the ternary operator.

To make this concrete, consider a case where only one branch performs a computation and that computation is straightforward. Under these conditions, an eager evaluation combined with SIMD typically outperforms a lazy alternative. In fact, this is precisely the strategy adopted when using a ternary operator.

IF

```
x              = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.296 ms (0 allocations: 0 bytes)
```

IFELSE

```
x              = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  412.153 μs (0 allocations: 0 bytes)
```

TERNARY

```
x              = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  312.681 μs (0 allocations: 0 bytes)
```

The situation reverses when the conditional branch involves a computationally intensive operation. Here, a lazy strategy avoids performing expensive work unnecessarily, leading to better performance overall. Below, we illustrate a scenario like this, showing that the ternary operator also chooses a lazy evaluation.

IF

```
x               = rand(2_000_000)
condition(a)    = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  39.390 ms (0 allocations: 0 bytes)
```

IFELSE

```
x               = rand(2_000_000)
condition(a)    = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  40.469 ms (0 allocations: 0 bytes)
```

TERNARY

```
x              = rand(2_000_000)
condition(a)   = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  40.580 ms (0 allocations: 0 bytes)
```

VECTORS WITH CONDITIONS

In many situations, you may already have a vector of Boolean conditions defined. This can happen because the condition vector is part of the dataset itself, or because the same conditions will be reused throughout the code, making it worthwhile to compute and store them in advance.

Julia represents such vectors using either `Vector{Bool}` or `BitVector`. The latter is the default output for broadcast expressions like `x .> 0`. While `BitVector` is memory-efficient and can be faster in some contexts, it can also inhibit SIMD optimizations. When this occurs, converting the `BitVector` to a `Vector{Bool}` can improve performance. The example below illustrates this effect: even after accounting for the cost of converting the type, the version using `Vector{Bool}` achieves a shorter execution time.

BITVECTOR

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  13.156 ms (3 allocations: 7.629 MiB)
```

VECTOR{BOOL}

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)
    boolvector = Vector(bitvector)

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  2.588 ms (5 allocations: 8.583 MiB)
```

No Vector Holding Conditions

When the vector of conditions isn't already available, the conclusions outlined above no longer apply. In those cases, the most efficient strategy to leverage SIMD is to avoid building a condition vector altogether. Instead, you should apply `ifelse` directly to scalar values inside the for-loop. This approach eliminates unnecessary memory allocations, while still allowing the compiler to generate SIMD instructions effectively. The following example demonstrates this point.

BITVECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    bitvector  = x .> 0.5


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  9.641 ms (7 allocations: 7.749 MiB)
```

VECTOR{BOOL}

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    boolvector = Vector{Bool}(undef,length(x))
        boolvector .= x .> 0.5

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  2.314 ms (5 allocations: 8.583 MiB)
```

NO VECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)



    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i]>0.5, x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  1.436 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC OPERATIONS AS COMPOUND CONDITIONS

Branches can often be eliminated by rewriting logical conditions in terms of simple algebraic identities. By doing so, we're eliminating the hurdle that impedes a direct application of SIMD.

To see how this works, let's first formalize Boolean logic in mathematical terms. Given a set $\{b_i\}_{i=1}^{n}$ where $b_i \in \{0, 1\}$:

- All conditions are satisfied when

$$\prod_{i=1}^{n} c_i = 1$$

- At least one condition is satisfied when

$$1 - \prod_{i=1}^{n} (1 - c_i) = 1$$

These identities translate directly into Boolean expressions. Specifically, given two Boolean scalars `c1` and `c2`:

- `c1 && c2` can be expressed as `Bool(c1 * c2)`,
- `c1 || c2` can be expressed as `Bool(1 - !c1 * !c2)`.

Once this strategy is adopted, control flow can be rewritten in a manner that's friendlier to vectorization. The following example illustrates this approach by replacing branching logic in a for-loop.

&&

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) && (y[i] < 0.6) && (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  6.708 ms (0 allocations: 0 bytes)
```

&& (ALGEBRAIC)

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3)  * (y[i] < 0.6)  * (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.924 ms (0 allocations: 0 bytes)
```

||

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) || (y[i] < 0.6) || (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  9.961 ms (0 allocations: 0 bytes)
```

|| (ALGEBRAIC)

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if Bool(1 - !(x[i] > 0.3)  * !(y[i] < 0.6)  * !(x[i] > y[i]))
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.806 ms (0 allocations: 0 bytes)
```

The following example does the same, but for the case of broadcasting.

&&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) && (y<0.6) && (x>y), x, y)
```

```
julia> foo($x,$y)
  15.562 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC &&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) *  (y<0.6) *  (x>y), x, y)
```

```
julia> foo($x,$y)
  1.748 ms (3 allocations: 7.629 MiB)
```

||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) || (y<0.6) || (x>y), x, y)
```

```
julia> foo($x,$y)
  11.352 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC ||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse(Bool(1 - !(x>0.3) * !(y<0.6) * !(x>y)), x, y)
```

```
julia> foo($x,$y)
  1.459 ms (3 allocations: 7.629 MiB)
```

Notice that such algebraic transformations may be performed automatically by the compiler. Whether this occurs depends on how the code is written. For example, revisiting the case of for-loops, the compiler performs the transformation automatically when expressing the condition through a separate function.

&&

```
x              = rand(1_000_000)
y              = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) && (y[i] < 0.6) && (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  6.732 ms (0 allocations: 0 bytes)
```

&& (ALGEBRAIC)

```
x              = rand(1_000_000)
y              = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3)  * (y[i] < 0.6)  * (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.727 ms (0 allocations: 0 bytes)
```

AUTOMATIC TRANSFORMATION

```
x              = rand(1_000_000)
y              = rand(1_000_000)
condition(a,b) = (a > 0.3)  * (b < 0.6)  * (a > b)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i],y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.791 ms (0 allocations: 0 bytes)
```

FOOTNOTES

[1] Recall that the compiler may automatically disable bounds checking in simple scenarios. For instance, this is the case when only `x` is indexed and `eachindex(x)` is employed as the iteration range. Instead, explicit use of `@inbounds` becomes necessary when both `x` and another vector indexed, as in the examples we introduce below.